

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 3

Procedimientos Almacenados y Triggers

Alumno: Tomás Rodeghiero
Año: 2026

Resolución completa sobre los motores *MySQL 8+*, *PostgreSQL 14+* y *Oracle XE*, apoyada en las bases creadas en los Prácticos 1 y 2.

Índice

1. Introducción	2
2. Ejercicio 1: Cantidad máxima de ítems por factura	2
2.1. MySQL 8+	2
2.2. PostgreSQL 14+	4
3. Ejercicio 2: Cotización máxima por banco (PostgreSQL)	6
4. Ejercicio 3: Saldo de cuenta bancaria (Oracle)	8
4.1. Inciso a) Procedimiento que inserta movimiento y mantiene saldo	9
4.2. Inciso b) Saldo a una fecha dada	10
5. Ejercicio 4: Detección de facturas inconsistentes (MySQL)	11
6. Ejercicio 5: Triggers y autonumerado	13
6.1. a) Mayúsculas antes de insertar un producto	13
6.2. b) Auditoría de cambios en <code>producto.cantidad</code>	14
6.3. c) Autonumerado de <code>parquimetro</code>	16
Conclusión	17

1. Introducción

El Práctico 3 introduce dos mecanismos centrales del lado servidor: **procedimientos y funciones almacenadas** y **triggers**. Ambos permiten desplazar lógica de negocio desde la aplicación hasta el motor de base de datos, garantizando reglas de integridad y de auditoría que se cumplen *siempre*, sin depender del cliente que ejecute la operación.

Los ejercicios reutilizan las bases de datos del Práctico 1 (inciso 1.a de facturación e inciso 1.b de estacionamiento) y, en el Ejercicio 4, los usuarios definidos en el Práctico 2. Cada inciso indica qué motor debe usarse:

- Ejercicio 1: MySQL 8+ y PostgreSQL 14+.
- Ejercicio 2: PostgreSQL 14+ exclusivamente.
- Ejercicio 3: Oracle XE 10/11.
- Ejercicio 4: MySQL 8+ con el usuario `vendedor2`.
- Ejercicio 5: el motor en el que originalmente se creó cada base (MySQL/PostgreSQL para facturación, MySQL/Oracle para estacionamiento).

Convenciones. Los nombres de bases siguen los del Práctico 1: `practico1a_mysql`, `practico1a` (esquema en PostgreSQL) y `practico1b_mysql`. En MySQL los procedimientos se rodean con `DELIMITER $$...$$ DELIMITER ;` para que el cliente no corte la sentencia en cada `;` interno.

2. Ejercicio 1: Cantidad máxima de ítems por factura

Enunciado. Sobre la base de facturación del Práctico 1.a, agregar a `factura` el campo `cantidad_maxima_items` y garantizar que ninguna factura supere ese máximo.

Estrategia

La consigna combina un cambio de estructura con una regla de negocio que no se puede expresar como un `CHECK` declarativo porque depende de *otra tabla*. Por eso se resuelve con tres elementos:

1. Un `ALTER TABLE` que agrega la columna y un `CHECK` sobre su rango.
2. Un trigger `BEFORE INSERT` (y `BEFORE UPDATE` cuando un ítem se reasigna a otra factura) sobre `item_factura`, que cuenta los ítems existentes y aborta si la inserción supera el límite.
3. Un trigger `BEFORE UPDATE` sobre `factura` que impide bajar `cantidad_maxima_items` por debajo de los ítems ya cargados.

2.1. MySQL 8+

```
USE practico1a_mysql;

-- 1) Agregar la columna con un default razonable y validacion de rango.
ALTER TABLE factura
  ADD COLUMN cantidad_maxima_items INT NOT NULL DEFAULT 10,
  ADD CONSTRAINT ck_factura_cantidad_maxima_items
    CHECK (cantidad_maxima_items > 0);

-- 2) Trigger BEFORE INSERT en item_factura.
DROP TRIGGER IF EXISTS bi_item_factura_control_max_items;
DELIMITER $$
CREATE TRIGGER bi_item_factura_control_max_items
```

```

BEFORE INSERT ON item_factura
FOR EACH ROW
BEGIN
    DECLARE v_max INT;
    DECLARE v_actual INT;

    SELECT f.cantidad_maxima_items INTO v_max
    FROM factura f
    WHERE f.nro_factura = NEW.nro_factura;

    SELECT COUNT(*) INTO v_actual
    FROM item_factura i
    WHERE i.nro_factura = NEW.nro_factura;

    IF v_actual + 1 > v_max THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La factura supera la cantidad maxima de items permitidos';
    END IF;
END$$
DELIMITER ;

-- 3) Trigger BEFORE UPDATE en item_factura: solo valida el cambio de
--     factura destino (mover un item de una factura a otra).
DROP TRIGGER IF EXISTS bu_item_factura_control_max_items;
DELIMITER $$
CREATE TRIGGER bu_item_factura_control_max_items
BEFORE UPDATE ON item_factura
FOR EACH ROW
BEGIN
    DECLARE v_max INT;
    DECLARE v_actual INT;

    IF NEW.nro_factura <> OLD.nro_factura THEN
        SELECT f.cantidad_maxima_items INTO v_max
        FROM factura f
        WHERE f.nro_factura = NEW.nro_factura;

        SELECT COUNT(*) INTO v_actual
        FROM item_factura i
        WHERE i.nro_factura = NEW.nro_factura;

        IF v_actual + 1 > v_max THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'La factura destino supera la cantidad maxima de items
permitidos';
        END IF;
    END IF;
END$$
DELIMITER ;

-- 4) Trigger BEFORE UPDATE en factura: impide bajar el maximo por
--     debajo de los items ya cargados.
DROP TRIGGER IF EXISTS bu_factura_control_max_items;
DELIMITER $$
CREATE TRIGGER bu_factura_control_max_items
BEFORE UPDATE ON factura
FOR EACH ROW
BEGIN
    DECLARE v_actual INT;

    IF NEW.cantidad_maxima_items <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'cantidad_maxima_items debe ser mayor que 0';
    END IF;
END$$
DELIMITER ;

```

```

END IF;

IF NEW.cantidad_maxima_items <> OLD.cantidad_maxima_items THEN
    SELECT COUNT(*) INTO v_actual
    FROM item_factura i
    WHERE i.nro_factura = OLD.nro_factura;

    IF v_actual > NEW.cantidad_maxima_items THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'No se puede reducir el maximo: ya hay mas items cargados';
    END IF;
END IF;
END$$
DELIMITER ;

```

¿Por qué tres triggers? El trigger de INSERT cubre la altas; el de UPDATE sobre `item_factura` sólo se activa cuando el ítem cambia de factura, evitando trabajo innecesario en updates de cantidad/precio; y el trigger de UPDATE sobre `factura` cubre el caso simétrico, en el que el problema no es el ítem que entra sino el límite que baja.

2.2. PostgreSQL 14+

PostgreSQL separa la *función* (que contiene la lógica) del *trigger* (que la dispara). Una sola función cubre INSERT y UPDATE usando TG_OP.

```

SET search_path TO practico1a;

-- 1) Agregar columna con default y CHECK.
ALTER TABLE practico1a.factura
    ADD COLUMN cantidad_maxima_items INTEGER NOT NULL DEFAULT 10;

ALTER TABLE practico1a.factura
    ADD CONSTRAINT ck_factura_cantidad_maxima_items
    CHECK (cantidad_maxima_items > 0);

-- 2) Funcion + trigger de control sobre item_factura.
DROP TRIGGER IF EXISTS biu_item_factura_control_max_items
ON practico1a.item_factura;
DROP FUNCTION IF EXISTS practico1a.fn_control_max_items_item_factura();

CREATE OR REPLACE FUNCTION practico1a.fn_control_max_items_item_factura()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_max INTEGER;
    v_actual INTEGER;
BEGIN
    -- En UPDATE de la misma factura no cambia la cantidad de items.
    IF TG_OP = 'UPDATE' AND NEW.nro_factura = OLD.nro_factura THEN
        RETURN NEW;
    END IF;

    SELECT f.cantidad_maxima_items INTO v_max
    FROM practico1a.factura f
    WHERE f.nro_factura = NEW.nro_factura
    FOR UPDATE;

    SELECT COUNT(*) INTO v_actual
    FROM practico1a.item_factura i
    WHERE i.nro_factura = NEW.nro_factura;

```

```

    IF v_actual + 1 > v_max THEN
        RAISE EXCEPTION
            'La factura % supera la cantidad maxima de items permitidos (%).',
            NEW.nro_factura, v_max;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER biu_item_factura_control_max_items
BEFORE INSERT OR UPDATE ON practico1a.item_factura
FOR EACH ROW
EXECUTE FUNCTION practico1a.fn_control_max_items_item_factura();

-- 3) Funcion + trigger de control sobre factura.cantidad_maxima_items.
DROP TRIGGER IF EXISTS bu_factura_control_max_items
ON practico1a.factura;
DROP FUNCTION IF EXISTS practico1a.fn_control_update_max_items_factura();

CREATE OR REPLACE FUNCTION practico1a.fn_control_update_max_items_factura()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_actual INTEGER;
BEGIN
    IF NEW.cantidad_maxima_items <= 0 THEN
        RAISE EXCEPTION 'cantidad_maxima_items debe ser mayor que 0';
    END IF;

    IF NEW.cantidad_maxima_items <> OLD.cantidad_maxima_items THEN
        SELECT COUNT(*) INTO v_actual
        FROM practico1a.item_factura i
        WHERE i.nro_factura = OLD.nro_factura;

        IF v_actual > NEW.cantidad_maxima_items THEN
            RAISE EXCEPTION
                'No se puede reducir el maximo de la factura %: tiene % items cargados',
                OLD.nro_factura, v_actual;
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER bu_factura_control_max_items
BEFORE UPDATE OF cantidad_maxima_items ON practico1a.factura
FOR EACH ROW
EXECUTE FUNCTION practico1a.fn_control_update_max_items_factura();

```

Detalles a destacar.

- En PostgreSQL se usa FOR UPDATE sobre la fila de factura para evitar condiciones de carrera bajo concurrencia: si dos sesiones intentan insertar el último ítem al mismo tiempo, una espera a que la otra confirme.
- El trigger BEFORE UPDATE OF cantidad_maxima_items sólo dispara cuando esa columna cambia, lo que es más eficiente que disparar en cualquier UPDATE.

Prueba (válida para ambos motores)

```
-- Suponiendo que la factura 1001 ya tiene 2 items cargados.
UPDATE factura SET cantidad_maxima_items = 2 WHERE nro_factura = 1001;

-- FALLA: dejaria 3 items en la factura 1001.
INSERT INTO item_factura (cod_producto, nro_factura, cantidad, precio)
VALUES (101, 1001, 1, 15000.00);

-- FALLA: intenta reducir el maximo por debajo de los 2 items existentes.
UPDATE factura SET cantidad_maxima_items = 1 WHERE nro_factura = 1001;

-- FUNCIONA: subir el maximo siempre es valido.
UPDATE factura SET cantidad_maxima_items = 5 WHERE nro_factura = 1001;
```

3. Ejercicio 2: Cotización máxima por banco (PostgreSQL)

Enunciado. Crear las tablas Banco y Cotizacion, e implementar una función que reciba un banco, calcule la cotización máxima *sin usar* MAX, actualice Banco.cotizacion_maxima y devuelva el resultado por un parámetro OUT.

Diseño

Como la consigna prohíbe MAX, la función recorre las cotizaciones del banco con un FOR ... LOOP en PL/pgSQL y mantiene una variable acumuladora con el mayor valor visto hasta el momento. Si el banco no tiene cotizaciones cargadas o no existe, la función levanta una excepción explícita.

```
CREATE SCHEMA IF NOT EXISTS practico3_ej2;
SET search_path TO practico3_ej2;

DROP TABLE IF EXISTS cotizacion;
DROP TABLE IF EXISTS banco;

CREATE TABLE banco (
    cod_banco INTEGER PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    precio NUMERIC(12,4) NOT NULL,
    cotizacion_maxima NUMERIC(12,4),
    CONSTRAINT ck_banco_precio CHECK (precio > 0),
    CONSTRAINT ck_banco_cot_max
        CHECK (cotizacion_maxima IS NULL OR cotizacion_maxima > 0)
);

CREATE TABLE cotizacion (
    cod_cotizacion INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cod_banco INTEGER NOT NULL,
    cotizacion NUMERIC(12,4) NOT NULL,
    CONSTRAINT ck_cotizacion_valor CHECK (cotizacion > 0),
    CONSTRAINT fk_cotizacion_banco
        FOREIGN KEY (cod_banco) REFERENCES banco(cod_banco)
        ON DELETE RESTRICT
);

CREATE INDEX idx_cotizacion_cod_banco ON cotizacion(cod_banco);
```

Función con parámetros IN/OUT

```
CREATE OR REPLACE FUNCTION practico3_ej2.fn_actualizar_cotizacion_maxima(
```

```

    IN p_cod_banco INTEGER,
    OUT p_cotizacion_maxima NUMERIC(12,4)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_cot NUMERIC(12,4);
BEGIN
    -- Verificacion de existencia + bloqueo del banco.
    PERFORM 1
        FROM practico3_ej2.banco b
        WHERE b.cod_banco = p_cod_banco
        FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe banco con cod_banco=%', p_cod_banco;
    END IF;

    p_cotizacion_maxima := NULL;

    -- Calculo manual del maximo, sin MAX.
    FOR v_cot IN
        SELECT c.cotizacion
        FROM practico3_ej2.cotizacion c
        WHERE c.cod_banco = p_cod_banco
    LOOP
        IF p_cotizacion_maxima IS NULL OR v_cot > p_cotizacion_maxima THEN
            p_cotizacion_maxima := v_cot;
        END IF;
    END LOOP;

    IF p_cotizacion_maxima IS NULL THEN
        RAISE EXCEPTION 'El banco % no tiene cotizaciones cargadas', p_cod_banco;
    END IF;

    -- Persistir el resultado en la tabla banco.
    UPDATE practico3_ej2.banco b
        SET cotizacion_maxima = p_cotizacion_maxima
        WHERE b.cod_banco = p_cod_banco;

    RETURN;
END;
$$;

```

Análisis.

- **Parámetros:** p_cod_banco entra como IN, p_cotizacion_maxima sale como OUT. PostgreSQL trata las funciones con OUT como devolviendo una fila con esas columnas; por eso se invoca con `SELECT * FROM fn...(...)`.
- **Estructura iterativa:** el `FOR ... LOOP` avanza una fila por iteración. La variable p_cotizacion_maxima se inicializa en NULL y se actualiza con cada valor mayor.
- **Manejo de errores:** dos `RAISE EXCEPTION` distintos para los dos casos de borde (banco inexistente y banco sin cotizaciones), con mensajes diferentes para que el cliente pueda diferenciarlos.
- **Concurrencia:** `PERFORM ... FOR UPDATE` bloquea la fila del banco para que dos invocaciones simultáneas no se pisen al actualizar cotizacion_maxima.

Datos de prueba y ejecución


```

SET search_path TO practico3_ej2;

INSERT INTO banco (cod_banco, nombre, precio, cotizacion_maxima) VALUES
  (1, 'Banco Rio', 1200.0000, NULL),
  (2, 'Banco Sur', 950.0000, NULL);

INSERT INTO cotizacion (cod_banco, cotizacion) VALUES
  (1, 1210.5000), (1, 1198.2000), (1, 1245.7500), (1, 1222.1000),
  (2, 960.0000), (2, 1001.3000), (2, 998.9000);

-- Llamado: la salida es 1245.7500 para el banco 1, 1001.3000 para el 2.
SELECT * FROM practico3_ej2.fn_actualizar_cotizacion_maxima(1);
SELECT * FROM practico3_ej2.fn_actualizar_cotizacion_maxima(2);

SELECT cod_banco, nombre, cotizacion_maxima
FROM practico3_ej2.banco
ORDER BY cod_banco;

```

4. Ejercicio 3: Saldo de cuenta bancaria (Oracle)

Enunciado. Crear las tablas `cuenta` y `movimiento` y dos procedimientos: (a) registrar un movimiento manteniendo el saldo y mostrarlo por pantalla, y (b) calcular el saldo de una cuenta a una fecha dada, mostrándolo por pantalla y devolviéndolo por un parámetro de salida.

Esquema

El saldo de cada cuenta se mantiene *denormalizado* en `cuenta.saldo` para evitar recalcularlo a partir de toda la historia en cada consulta. La consistencia entre ambos se mantiene mediante el procedimiento de inserción.

```

SET SERVEROUTPUT ON;

-- Tablas
CREATE TABLE cuenta (
  nro_cuenta NUMBER(10) CONSTRAINT pk_cuenta PRIMARY KEY,
  saldo NUMBER(14,2) DEFAULT 0 NOT NULL
);

CREATE TABLE movimiento (
  nro_movimiento NUMBER(12) CONSTRAINT pk_movimiento PRIMARY KEY,
  nro_cuenta NUMBER(10) NOT NULL,
  fecha DATE DEFAULT SYSDATE NOT NULL,
  debe NUMBER(14,2) DEFAULT 0 NOT NULL,
  haber NUMBER(14,2) DEFAULT 0 NOT NULL,
  CONSTRAINT fk_movimiento_cuenta
    FOREIGN KEY (nro_cuenta) REFERENCES cuenta(nro_cuenta),
  CONSTRAINT ck_mov_montos_no_negativos
    CHECK (debe >= 0 AND haber >= 0),
  CONSTRAINT ck_mov_operacion_valida
    CHECK ((debe > 0 AND haber = 0) OR (haber > 0 AND debe = 0))
);

-- En Oracle 10g/11g no hay IDENTITY: secuencia + trigger.
CREATE SEQUENCE seq_movimiento
  START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;

CREATE OR REPLACE TRIGGER bi_movimiento
BEFORE INSERT ON movimiento
FOR EACH ROW

```

```

BEGIN
  IF :NEW.nro_movimiento IS NULL THEN
    SELECT seq_movimiento.NEXTVAL INTO :NEW.nro_movimiento FROM dual;
  END IF;

  IF :NEW.fecha IS NULL THEN
    :NEW.fecha := SYSDATE;
  END IF;
END;
/

```

4.1. Inciso a) Procedimiento que inserta movimiento y mantiene saldo

```

CREATE OR REPLACE PROCEDURE sp_registrar_movimiento (
  p_nro_cuenta IN cuenta.nro_cuenta%TYPE,
  p_debe       IN NUMBER,
  p_haber      IN NUMBER
)
IS
  v_saldo_actual cuenta.saldo%TYPE;
  v_debe  NUMBER(14,2) := NVL(p_debe, 0);
  v_haber NUMBER(14,2) := NVL(p_haber, 0);
BEGIN
  -- Validaciones de los parametros.
  IF v_debe < 0 OR v_haber < 0 THEN
    RAISE_APPLICATION_ERROR(-20001,
      'Debe/Haber no pueden ser negativos');
  END IF;

  IF NOT ((v_debe > 0 AND v_haber = 0)
    OR (v_haber > 0 AND v_debe = 0)) THEN
    RAISE_APPLICATION_ERROR(-20002,
      'La operacion debe tener solo DEBE o solo HABER');
  END IF;

  -- Bloqueo de la cuenta para mantener consistencia bajo concurrencia.
  SELECT c.saldo INTO v_saldo_actual
    FROM cuenta c
   WHERE c.nro_cuenta = p_nro_cuenta
   FOR UPDATE;

  INSERT INTO movimiento (nro_cuenta, fecha, debe, haber)
  VALUES (p_nro_cuenta, SYSDATE, v_debe, v_haber);

  UPDATE cuenta c
    SET c.saldo = c.saldo + v_debe - v_haber
   WHERE c.nro_cuenta = p_nro_cuenta
   RETURNING c.saldo INTO v_saldo_actual;

  DBMS_OUTPUT.PUT_LINE(
    'Cuenta: ' || p_nro_cuenta || ' | Saldo actual: ' ||
    TO_CHAR(v_saldo_actual, 'FM9999999990D00')
  );
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20003,
      'No existe la cuenta ' || p_nro_cuenta);
END;
/

```

Lógica del procedimiento.

1. Recibe los parámetros IN: p_nro_cuenta, p_debe, p_haber.
2. Reemplaza eventuales NULL por 0 con NVL y valida que la operación sea *sólo* debe o *sólo* haber.
3. Bloquea la fila de la cuenta con SELECT ... FOR UPDATE para evitar condiciones de carrera entre dos movimientos concurrentes sobre la misma cuenta.
4. Inserta el movimiento (la columna nro_movimiento se completa automáticamente con el trigger bi_movimiento).
5. Actualiza el saldo en la misma transacción y captura el nuevo valor con RETURNING.
6. Imprime el resultado con DBMS_OUTPUT.PUT_LINE.
7. En caso de cuenta inexistente, atrapa NO_DATA_FOUND y emite un error de aplicación diferenciado.

4.2. Inciso b) Saldo a una fecha dada

```
CREATE OR REPLACE PROCEDURE sp_saldo_a_fecha (
    p_nro_cuenta IN  cuenta.nro_cuenta%TYPE,
    p_fecha      IN  DATE,
    p_saldo      OUT NUMBER
)
IS
    v_saldo_actual    cuenta.saldo%TYPE;
    v_delta_posterior NUMBER(14,2);
BEGIN
    SELECT c.saldo INTO v_saldo_actual
    FROM cuenta c
    WHERE c.nro_cuenta = p_nro_cuenta;

    -- Suma del impacto de los movimientos POSTERIORES a la fecha.
    SELECT NVL(SUM(m.debe - m.haber), 0)
    INTO v_delta_posterior
    FROM movimiento m
    WHERE m.nro_cuenta = p_nro_cuenta
    AND m.fecha > p_fecha;

    p_saldo := v_saldo_actual - v_delta_posterior;

    DBMS_OUTPUT.PUT_LINE(
        'Cuenta: ' || p_nro_cuenta ||
        ' | Fecha: ' || TO_CHAR(p_fecha, 'YYYY-MM-DD HH24:MI:SS') ||
        ' | Saldo: ' || TO_CHAR(p_saldo, 'FM9999999990D00')
    );
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20004,
            'No existe la cuenta ' || p_nro_cuenta);
END;
/
```

Idea del cálculo. Se parte del saldo actual y se le resta el efecto neto de los movimientos cuya fecha sea estrictamente mayor a p_fecha: así se obtiene cuánto valía la cuenta al *cierre* de p_fecha. La alternativa — sumar todos los movimientos hasta esa fecha desde el comienzo — es equivalente pero más cara cuando hay muchos movimientos antiguos.

Prueba

```

SET SERVEROUTPUT ON;

INSERT INTO cuenta (nro_cuenta, saldo) VALUES (1001, 0);
INSERT INTO cuenta (nro_cuenta, saldo) VALUES (1002, 500);

BEGIN
    sp_registrar_movimiento(1001, 1000, 0); -- saldo 1000
    sp_registrar_movimiento(1001, 0, 150);  -- saldo 850
    sp_registrar_movimiento(1001, 200, 0);  -- saldo 1050
END;
/

DECLARE
    v_saldo NUMBER;
BEGIN
    sp_saldo_a_fecha(1001, SYSDATE, v_saldo);
    DBMS_OUTPUT.PUT_LINE(
        'Saldo retornado por OUT: ' ||
        TO_CHAR(v_saldo, 'FM9999999990D00'));
END;
/

```

5. Ejercicio 4: Detección de facturas inconsistentes (MySQL)

Enunciado. Crear un procedimiento, en MySQL, que detecte facturas *inconsistentes* (cuando la suma de $\text{cantidad} \times \text{precio}$ de sus ítems no coincide con `factura.monto`) y las almacene en una tabla. El procedimiento debe ser creado con el usuario `vendedor2` del Práctico 2 y ejecutarse con permisos del *invocador*.

Preparación (como root)

En el Práctico 2 (inciso 1.f) se le revocaron todos los privilegios a `vendedor2`. Antes de poder crear el procedimiento como `vendedor2`, hay que reotorgarle los permisos mínimos y, dado que el procedimiento usará `SQL SECURITY INVOKER`, también hay que dotar de los privilegios necesarios a `vendedor1` (el otro usuario que lo invocará).

```

USE practico1a_mysql;

-- Tabla destino donde se guardan las inconsistencias detectadas.
CREATE TABLE IF NOT EXISTS facturas_inconsistentes (
    nro_factura      INT PRIMARY KEY,
    monto_factura    DECIMAL(12,2) NOT NULL,
    monto_items      DECIMAL(12,2) NOT NULL,
    diferencia        DECIMAL(12,2) NOT NULL,
    fecha_control    DATETIME NOT NULL,
    usuario_ejecucion VARCHAR(100) NOT NULL
);

-- Permiso minimo para que vendedor2 pueda crear procedimientos.
GRANT CREATE ROUTINE ON practico1a_mysql.*
    TO 'vendedor2'@'localhost';

-- Como el procedimiento corre con SQL SECURITY INVOKER,
-- cada usuario que lo ejecute necesita estos permisos:
GRANT SELECT (nro_factura, monto)
    ON practico1a_mysql.factura
    TO 'vendedor1'@'localhost', 'vendedor2'@'localhost';

GRANT SELECT (nro_factura, cantidad, precio)

```

```

ON practico1a_mysql.item_factura
TO 'vendedor1'@'localhost', 'vendedor2'@'localhost';

GRANT INSERT, UPDATE, SELECT
ON practico1a_mysql.facturas_inconsistentes
TO 'vendedor1'@'localhost', 'vendedor2'@'localhost';

```

Creación del procedimiento (conectado como vendedor2)

```

-- $ mysql -u vendedor2 -p practico1a_mysql

DROP PROCEDURE IF EXISTS sp_guardar_facturas_inconsistentes;
DELIMITER $$
CREATE PROCEDURE sp_guardar_facturas_inconsistentes()
SQL SECURITY INVOKER
BEGIN
    INSERT INTO facturas_inconsistentes (
        nro_factura, monto_factura, monto_items,
        diferencia, fecha_control, usuario_ejecucion
    )
    SELECT
        f.nro_factura,
        f.monto AS monto_factura,
        ROUND(COALESCE(SUM(i.cantidad * i.precio), 0), 2) AS monto_items,
        ROUND(COALESCE(SUM(i.cantidad * i.precio), 0) - f.monto, 2)
            AS diferencia,
        NOW() AS fecha_control,
        USER() AS usuario_ejecucion
    FROM factura f
    LEFT JOIN item_factura i ON i.nro_factura = f.nro_factura
    GROUP BY f.nro_factura, f.monto
    HAVING ROUND(COALESCE(SUM(i.cantidad * i.precio), 0), 2)
        <> ROUND(f.monto, 2)
    ON DUPLICATE KEY UPDATE
        monto_factura      = VALUES(monto_factura),
        monto_items        = VALUES(monto_items),
        diferencia          = VALUES(diferencia),
        fecha_control       = VALUES(fecha_control),
        usuario_ejecucion = VALUES(usuario_ejecucion);
END$$
DELIMITER ;

```

Por qué SQL SECURITY INVOKER. Por defecto MySQL crea procedimientos con SQL SECURITY DEFINER, lo que hace que todas las ejecuciones se realicen con los permisos de quien lo creó. La consigna pide lo contrario: que el procedimiento corra con los permisos de quien lo invoca. Eso obliga a que *cada* invocador posea los permisos que la rutina necesita; de ahí los GRANT de la sección anterior.

Verificación del DEFINER. Para confirmar que se creó como vendedor2 y con SQL SECURITY INVOKER:

```

SHOW CREATE PROCEDURE sp_guardar_facturas_inconsistentes\G
-- En la salida debe verse:
-- DEFINER = 'vendedor2'@'localhost'
-- SQL SECURITY INVOKER

```

Permiso de ejecución (como root)

```
GRANT EXECUTE ON PROCEDURE practico1a_mysql.sp_guardar_facturas_inconsistentes
TO 'vendedor1'@'localhost', 'vendedor2'@'localhost';
```

Prueba con ambos usuarios

Para forzar una inconsistencia (sólo root suele tener UPDATE sobre factura):

```
USE practico1a_mysql;
UPDATE factura SET monto = monto + 100 WHERE nro_factura = 1001;
```

Luego, conectándose como cada usuario y ejecutando:

```
-- $ mysql -u vendedor2 -p practico1a_mysql
CALL sp_guardar_facturas_inconsistentes();
SELECT * FROM facturas_inconsistentes
ORDER BY fecha_control DESC, nro_factura;

-- $ mysql -u vendedor1 -p practico1a_mysql
CALL sp_guardar_facturas_inconsistentes();
SELECT * FROM facturas_inconsistentes
ORDER BY fecha_control DESC, nro_factura;
```

¿Por qué ON DUPLICATE KEY UPDATE? Porque nro_factura es PRIMARY KEY en facturas_inconsistentes: si una factura ya estaba marcada como inconsistente y se vuelve a ejecutar el procedimiento, en lugar de fallar por clave duplicada se actualiza con la última lectura. La columna usuario_ejecucion usa USER(), que en MySQL devuelve el usuario realmente conectado, lo que permite distinguir quién corrió el control.

6. Ejercicio 5: Triggers y autonumerado

6.1. a) Mayúsculas antes de insertar un producto

Se trata de un trigger BEFORE INSERT que normaliza el dato antes de guardarlo, de modo que la auditoría posterior y las búsquedas queden uniformes.

MySQL 8+ (practico1a_mysql)

```
USE practico1a_mysql;

DROP TRIGGER IF EXISTS bi_producto_mayusculas;
DELIMITER $$
CREATE TRIGGER bi_producto_mayusculas
BEFORE INSERT ON producto
FOR EACH ROW
BEGIN
    SET NEW.descripcion = UPPER(TRIM(NEW.descripcion));
END$$
DELIMITER ;
```

PostgreSQL 14+ (practico1a)

```
SET search_path TO practico1a;

DROP TRIGGER IF EXISTS bi_producto_mayusculas ON practico1a.producto;
DROP FUNCTION IF EXISTS practico1a.fn_producto_mayusculas();
```

```

CREATE OR REPLACE FUNCTION practico1a.fn_producto_mayusculas()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.descripcion := UPPER(BTRIM(NEW.descripcion));
    RETURN NEW;
END;
$$;

CREATE TRIGGER bi_producto_mayusculas
BEFORE INSERT ON practico1a.producto
FOR EACH ROW
EXECUTE FUNCTION practico1a.fn_producto_mayusculas();

```

Detalle. En MySQL se asigna `NEW.descripcion` con `SET`; en PostgreSQL se asigna directamente con `:=` y se devuelve `NEW`. El `TRIM/BTRIM` adicional limpia espacios al principio y al final, defendiéndose de errores de formato.

6.2. b) Auditoría de cambios en `producto.cantidad`

La auditoría debe registrar `cod_producto`, el *movimiento* (`NEW.cantidad - OLD.cantidad`), la fecha y el *usuario logueado en la aplicación cliente*. Como el motor no tiene forma de conocer el usuario de aplicación por sí mismo, la aplicación lo provee a través de una variable de sesión que el trigger lee.

MySQL 8+

```

USE practico1a_mysql;

CREATE TABLE IF NOT EXISTS auditoriaProducto (
    id_auditoria        BIGINT AUTO_INCREMENT PRIMARY KEY,
    cod_producto        INT NOT NULL,
    movimiento          INT NOT NULL,
    fecha_actualizacion DATETIME NOT NULL,
    usuario_aplicacion  VARCHAR(100) NOT NULL,
    CONSTRAINT fk_auditoria_producto
        FOREIGN KEY (cod_producto) REFERENCES producto(cod_producto)
);

DROP TRIGGER IF EXISTS au_producto_auditoria_cantidad;
DELIMITER $$
CREATE TRIGGER au_producto_auditoria_cantidad
AFTER UPDATE ON producto
FOR EACH ROW
BEGIN
    DECLARE v_usuario_app VARCHAR(100);

    IF NEW.cantidad <> OLD.cantidad THEN
        -- La aplicacion debe setear esta variable por sesion.
        -- SET @app_user_code = 'VENDEDOR_APP_01';
        SET v_usuario_app = @app_user_code;

        IF v_usuario_app IS NULL OR TRIM(v_usuario_app) = '' THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Debe definir @app_user_code en la sesion para auditar';
        END IF;
    END IF;

```

```

INSERT INTO auditoriaProducto (
    cod_producto, movimiento,
    fecha_actualizacion, usuario_aplicacion
)
VALUES (
    NEW.cod_producto,
    NEW.cantidad - OLD.cantidad,
    NOW(),
    v_usuario_app
);
END IF;
END$$
DELIMITER ;

```

Uso desde la aplicación.

```

SET @app_user_code = 'VENDEDOR_APP_01';
UPDATE producto SET cantidad = cantidad + 5 WHERE cod_producto = 101;

SELECT * FROM auditoriaProducto ORDER BY id_auditoria DESC;

```

PostgreSQL 14+

```

SET search_path TO practico1a;

CREATE TABLE IF NOT EXISTS practico1a.auditoria_producto (
    id_auditoria          BIGSERIAL PRIMARY KEY,
    cod_producto          INTEGER NOT NULL
        REFERENCES practico1a.producto(cod_producto),
    movimiento            INTEGER NOT NULL,
    fecha_actualizacion   TIMESTAMP NOT NULL,
    usuario_aplicacion    VARCHAR(100) NOT NULL
);

DROP TRIGGER IF EXISTS au_producto_auditoria_cantidad
ON practico1a.producto;
DROP FUNCTION IF EXISTS practico1a.fn_auditoria_producto_cantidad();

CREATE OR REPLACE FUNCTION practico1a.fn_auditoria_producto_cantidad()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_usuario_app TEXT;
BEGIN
    IF NEW.cantidad <> OLD.cantidad THEN
        -- La aplicacion debe setear esta variable de sesion.
        -- SET app.user_code = 'VENDEDOR_APP_01';
        v_usuario_app := current_setting('app.user_code', true);

        IF v_usuario_app IS NULL OR BTRIM(v_usuario_app) = '' THEN
            RAISE EXCEPTION
                'Debe definir app.user_code en la sesion para auditar';
        END IF;

        INSERT INTO practico1a.auditoria_producto (
            cod_producto, movimiento,
            fecha_actualizacion, usuario_aplicacion
        )
        VALUES (
            NEW.cod_producto,
            NEW.cantidad - OLD.cantidad,

```



```

        NOW(),
        v_usuario_app
    );
END IF;

RETURN NEW;
END;
$$;

CREATE TRIGGER au_producto_auditoria_cantidad
AFTER UPDATE OF cantidad ON practico1a.producto
FOR EACH ROW
EXECUTE FUNCTION practico1a.fn_auditoria_producto_cantidad();

```

Por qué AFTER UPDATE y no BEFORE. La auditoría sólo tiene sentido cuando la modificación ya ocurrió. Si algún CHECK o trigger anterior aborta el UPDATE, no debe haber registro de auditoría: usando AFTER UPDATE, PostgreSQL y MySQL garantizan que el trigger sólo dispara cuando la operación es exitosa.

6.3. c) Autonumerado de parquimetro

Sobre la base practico1b (estacionamiento del Práctico 1.b) hay que hacer que `id_parquimetro` se asigne automáticamente.

MySQL 8+

```

USE practico1b_mysql;

ALTER TABLE parquimetro
MODIFY id_parquimetro INT NOT NULL AUTO_INCREMENT;

```

Después de ejecutar esto, basta con omitir la columna en el INSERT:

```

INSERT INTO parquimetro (calle, altura) VALUES ('Colon', 1500);
SELECT * FROM parquimetro ORDER BY id_parquimetro;

```

Oracle XE 10/11 (SEQUENCE + TRIGGER)

Oracle 10/11 XE no tiene IDENTITY (apareció en Oracle 12c), así que el patrón clásico es *secuencia + trigger*. La secuencia genera valores únicos crecientes; el trigger los asigna a `:NEW.id_parquimetro` cuando la sentencia no provee uno explícito.

```

-- Limpieza opcional para re-ejecutar el script.
BEGIN EXECUTE IMMEDIATE 'DROP TRIGGER bi_parquimetro_autonumerado';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE seq_parquimetro';
EXCEPTION WHEN OTHERS THEN NULL; END;
/

-- Crear la secuencia arrancando despues del MAX existente,
-- para no chocar con datos cargados.
DECLARE
    v_start NUMBER;
BEGIN
    SELECT NVL(MAX(id_parquimetro), 0) + 1
    INTO v_start

```

```
FROM parquemetro;

EXECUTE IMMEDIATE
  'CREATE SEQUENCE seq_parquemetro START WITH ' || v_start ||
  ' INCREMENT BY 1 NOCACHE NOCYCLE';
END;
/

CREATE OR REPLACE TRIGGER bi_parquemetro_autonumerado
BEFORE INSERT ON parquemetro
FOR EACH ROW
WHEN (NEW.id_parquemetro IS NULL)
BEGIN
  SELECT seq_parquemetro.NEXTVAL
  INTO :NEW.id_parquemetro
  FROM dual;
END;
/
```

Prueba:

```
INSERT INTO parquemetro (calle, altura) VALUES ('Colon', 1500);
SELECT * FROM parquemetro ORDER BY id_parquemetro;
```

Comparación con MySQL. En MySQL es suficiente declarar la columna como `AUTO_INCREMENT`. En Oracle 10/11 XE el mismo efecto requiere dos objetos cooperando: la secuencia (`seq_parquemetro`) que produce valores y el trigger (`bi_parquemetro_autonumerado`) que los inyecta en la fila sólo cuando el cliente no provee uno explícito. La cláusula `WHEN (NEW.id_parquemetro IS NULL)` evita pisar los inserts con ID explícito — típicos en cargas de datos de prueba.

Conclusión

Los cinco ejercicios cubren el espectro de programación del lado servidor: *triggers* para reglas que dependen de varias tablas (Ej. 1), *funciones* con cálculo manual y parámetros IN/OUT (Ej. 2), *procedimientos* con validación, transaccionalidad y mensajes por pantalla (Ej. 3), *procedimientos* con seguridad por invocador (Ej. 4) y *triggers* de normalización, auditoría y autonumerado (Ej. 5).

Los puntos importantes para llevarse al parcial son:

- **BEFORE/AFTER + INSERT/UPDATE/DELETE** es la matriz básica del trigger; `NEW` y `OLD` dan acceso a la fila nueva y vieja.
- Para *abortar* desde el trigger: `SIGNAL SQLSTATE '45000'` en MySQL, `RAISE EXCEPTION` en PostgreSQL, `RAISE_APPLICATION_ERROR` en Oracle.
- Las *funciones* se asocian a `RETURN` de un valor; los *procedimientos* se invocan con `CALL` (MySQL, PostgreSQL) o en bloques `BEGIN ... END` (Oracle) y pueden tener parámetros `IN`, `OUT` e `INOUT`.
- Para *auditar el usuario de aplicación* se usan variables de sesión: `@var` en MySQL, `current_setting(...)` en PostgreSQL.
- `SQL SECURITY INVOKER/DEFINER` en MySQL es la pieza que conecta el Práctico 3 con el Práctico 2 (DCL): el primero asume permisos del invocador, el segundo (default) los del creador.
- `AUTO_INCREMENT` (MySQL) y `IDENTITY` (PostgreSQL/Oracle 12+) ahorran código; en Oracle 10/11 XE el patrón es `SEQUENCE + TRIGGER BEFORE INSERT`.